Manual de instalação e configuração

Backend do Katch — versão m3-v01 (tag `v0.3.0`)

Projeto	Katch — recrutamento e emparelhamento entre Candidatos e Empresas
Unidade curricular	Laboratório de Desenvolvimento de Software (LDS), LEI, 2026/2027, turma LEI3T2, grupo 11
Documento	`m3-manual-instalacao-configuracao-v01.pdf`
Componente abrangida	Backend (API ASP.NET Core e base de dados PostgreSQL). Os frontends web e mobile são acrescentados na versão de m4.
Versão do backend	**m3-v01**: tag Git `v0.3.0`, commit `3f4a070946035440e8ad9c746b83a7b904478881` (main de 2026-11-02 15:30 UTC) do repositório `katch-lds/katch-backend`. Cópia arquivada em `projeto/04-artefactos-tecnicos/04.12-backend/m3-v01`.
Issue	I106 (Sprint 06)
Fontes	`README.md` e `database/README.md` do repositório na versão entregue; Regulamento da UC, secção 19 (04.12, 04.13 e 04.17)

Histórico de versões

Versão	Data	Alteração	Issue
v01	2026-11-02	Primeira versão, para o backend m3-v01. Redigida a partir do README do repositório, com os valores atualizados para a versão entregue (número de testes, endpoints, variáveis) e com os passos executados de raiz num ambiente limpo (anexo A).	I106

Índice

1. Objetivo e público-alvo
2. Pré-requisitos
3. Instalação das ferramentas
4. Obter o código do backend
5. Criar a base de dados
6. Variáveis de ambiente
7. Criar o esquema e os dados iniciais
8. Arrancar a API
9. Verificação
10. Configuração opcional
11. Resolução de problemas
12. Resumo dos comandos
13. Anexo A — Ensaio num ambiente limpo

1. Objetivo e público-alvo

Este manual explica como instalar, configurar, arrancar e verificar o backend do Katch numa máquina Windows, sem ajuda da equipa. Está escrito para o docente e para quem precise de pôr a API a funcionar de raiz. No fim deve ter:

- a base de dados `katch` no PostgreSQL 18, com as 17 tabelas do modelo de dados v02, a conta de Administrador e as listas pré-definidas;
- a API a responder em `http://localhost:5180`, com a documentação interativa (Swagger) em `http://localhost:5180/swagger`;
- os 2000 testes automáticos (1229 unitários e 771 de integração) a passar.

Todos os comandos são para o **Windows PowerShell** (o prompt começa por `PS`). Se a janela mostrar `C:\...>` sem `PS`, é a Linha de Comandos (cmd): escreva `powershell` e carregue em Enter. Os valores entre `< >` são para substituir pelos seus.

2. Pré-requisitos

Requisito	Versão	Notas
Sistema operativo	Windows 10 ou 11, 64 bits	Com Windows PowerShell 5.1 (incluído no Windows). Ensaio feito em Windows 11 Home 10.0.26200.
SDK do .NET	**10.0.401 ou superior** (banda 10.0.4xx)	Fixado em `global.json` (`"version": "10.0.401"`, `rollForward: latestFeature`). O runtime sozinho não chega. Ensaio feito com o 10.0.402.
PostgreSQL	**18**	Inclui o `psql`, o `createdb` e o pgAdmin. A base tem de usar a codificação UTF8 (a predefinida do instalador). Ensaio feito com o 18.3.
Git	Qualquer versão recente	Só é preciso para obter o código do GitHub (secção 4, opção B). Ensaio feito com o 2.55.

Ligação à Internet	—	Na primeira compilação, o `dotnet` descarrega os pacotes NuGet (cerca de 1 minuto). Depois disso não é precisa.
Portos livres	5432 (ou outro) e 5180	5432 para o PostgreSQL (o instalador propõe outro se já estiver ocupado); 5180 (HTTP) e 7180 (HTTPS) para a API.

A ferramenta `dotnet-ef` 10.0.13 (migrations) não é instalada à parte: vem no manifesto `.config/dotnet-tools.json` do repositório e é instalada com `dotnet tool restore` (secção 7.1). As versões dos pacotes NuGet estão em `Directory.Packages.props` (EF Core 10.0.13, Npgsql 10.0.3, Swashbuckle 10.3.0, xUnit v3 3.2.2).

3. Instalação das ferramentas

Se alguma ferramenta já estiver instalada na versão indicada, passe à seguinte. Os comandos `winget` correm no PowerShell; também pode usar os instaladores dos sítios oficiais. Em 2026-11-02 o `winget` instalava o SDK 10.0.403, o PostgreSQL 18.6 e o Git 2.56, todos compatíveis.

3.1. SDK do .NET 10

```
winget install Microsoft.DotNet.SDK.10
```

Ou descarregue o instalador do SDK (não do runtime) em `https://dotnet.microsoft.com/download/dotnet/10.0`. Depois **feche e abra o PowerShell**, porque o PATH só é atualizado em janelas novas, e confirme:

```
dotnet --list-sdks
```

Tem de aparecer uma linha `10.0.401` ou superior (por exemplo, `10.0.402 [C:\Program Files\dotnet\sdk]`).

3.2. PostgreSQL 18

```
winget install PostgreSQL.PostgreSQL.18
```

Ou use o instalador da EDB em `https://www.postgresql.org/download/windows/`. No assistente:

1. Mantenha os componentes *PostgreSQL Server*, *pgAdmin 4* e *Command Line Tools*.
2. Defina e **anote** a palavra-passe do superutilizador `postgres`.
3. Anote o **porto** (5432 por omissão; se já houver outro PostgreSQL, o instalador propõe outro, por exemplo 5433).
4. Mantenha a *Locale* predefinida (*[Default locale]*). O instalador cria o servidor com a codificação UTF8.

O instalador não põe o `psql` no PATH. Para o usar nesta janela:

```
$env:Path = "C:\Program Files\PostgreSQL\18\bin;" + $env:Path
psql --version
```

Deve mostrar `psql (PostgreSQL) 18.x`. O serviço aparece em *Serviços* do Windows como `postgresql-x64-18`.

3.3. Git (opcional)

```
winget install Git.Git
git --version
```

4. Obter o código do backend

Há duas formas, com o mesmo resultado (a mesma árvore de ficheiros):

Opção A — a partir da entrega. Copie a pasta `projeto/04-artefactos-tecnicos/04.12-backend/m3-v01` para uma pasta de caminho curto, por exemplo `C:\katch\katch-backend`.

Opção B — a partir do GitHub (repositório `katch-lds/katch-backend`, com acesso dado ao docente):

```
cd C:\
git clone https://github.com/katch-lds/katch-backend.git C:\katch\katch-backend
cd C:\katch\katch-backend
git checkout v0.3.0
git log -1 --format="%H"
```

O último comando tem de mostrar `3f4a070946035440e8ad9c746b83a7b904478881`.

Use uma pasta de caminho curto. O ficheiro com o nome mais longo tem 108 caracteres (`src/Katch.Backend/Infrastructure/Persistence/Migrations/20261024180929_AddMessageAndOperationLog.Designer.cs`). Se a pasta escolhida tiver mais de cerca de 150 caracteres, o Windows ultrapassa o limite de 260 e o `git clone` falha com `Filename too long` (aconteceu no ensaio do anexo A). Ver a secção 11.

A partir daqui, **todos os comandos correm na raiz do repositório** (a pasta onde está o `Katch.Backend.sln`):

Pasta ou ficheiro	Conteúdo
`src/Katch.Backend/`	A API: controllers, serviços, repositórios, entidades, tarefas periódicas e migrations
`database/`	Scripts SQL de criação, reposição, dados iniciais e verificação (também em `04.13-scripts-de-base-de-dados/m3-v01`)
`tests/Katch.Backend.UnitTests/`	Testes unitários (xUnit v3)
`tests/Katch.Backend.IntegrationTests/`	Testes de integração (xUnit v3, SQLite em memória)
`Katch.Backend.sln`, `global.json`, `Directory.*.props`	Solução, versão do SDK, regras de compilação e versões dos pacotes
`.config/dotnet-tools.json`	Ferramenta local `dotnet-ef`
`.github/workflows/ci.yml`	Pipeline CI (compilação, formatação, testes, scripts SQL sobre PostgreSQL 18)

5. Criar a base de dados

Crie uma base vazia chamada `katch`. As tabelas são criadas na secção 7.

Com o pgAdmin: ligue-se ao servidor (Host `localhost`, o porto e o utilizador `postgres` da secção 3.2); clique com o botão direito em *Databases* > *Create* > *Database…*; em *Database* escreva `katch` e clique em *Save*.

Ou com o PowerShell (o `psql` e o `createdb` usam as variáveis `PG*`):

```
$env:PGHOST = "localhost"
$env:PGPORT = "<porto>"
$env:PGUSER = "postgres"
$env:PGPASSWORD = "<palavra-passe do postgres>"
createdb katch
psql -d katch -Atc "SELECT pg_encoding_to_char(encoding), datctype FROM pg_database WHERE datname = 'katch';"
```

O resultado tem de começar por `UTF8` (por exemplo, `UTF8|Portuguese_Portugal.1252`). Não use a locale `C`: o CHECK das competências «Outro» usa `[[:alnum:]]` e, com a locale `C`, recusa letras acentuadas.

6. Variáveis de ambiente

O backend não tem credenciais em ficheiros versionados: o `appsettings.json` só configura os registos de atividade. Tudo o que é secreto ou depende da máquina vem de variáveis de ambiente (nomes com **dois sublinhados**, que correspondem às chaves de configuração com `:`).

Variável	Obrigatória para	Valor e regras
`ConnectionStrings__Katch`	API, aplicar e reverter migrations	`Host=localhost;Port=<porto>;Database=katch;Username=postgres;Password=<palavra-passe>`. Sem ela, a API não arranca.
`Jwt__Secret`	API	Segredo que assina as credenciais de sessão (JWT HS256), com **pelo menos 32 bytes**. Sem ele, ou mais curto, a API não arranca.
`Katch__AdminPassword`	Aplicar as migrations numa base nova	Palavra-passe da conta de Administrador `admin@katch.pt`, criada pela migration `AddAdmin`. Pelo menos 8 caracteres, com uma letra e um algarismo (P05). Na base fica só o resumo.
`BackgroundJobs__IntervalSeconds`	Opcional	Intervalo das tarefas periódicas (reposição da quota de interesses e encerramento das vagas expiradas), de 1 a 86400; 60 por omissão.
`FileStorage__RootPath`	Opcional	Pasta das fotografias, logótipos e CV enviados; por omissão `%LOCALAPPDATA%\Katch\files`.

Defina as três primeiras na janela do PowerShell onde vai trabalhar:

```
$env:ConnectionStrings__Katch = "Host=localhost;Port=<porto>;Database=katch;Username=postgres;Password=<palavra-passe do post
gres>"
$env:Katch__AdminPassword = "<palavra-passe do Administrador, por exemplo com 12 caracteres>"
$env:Jwt__Secret = "<segredo com pelo menos 32 caracteres>"
```

Para gerar um segredo aleatório com 64 caracteres:

```
$b = New-Object byte[] 48; [Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
$env:Jwt__Secret = [Convert]::ToBase64String($b)
```

> **As variáveis `$env:` só existem na janela onde foram definidas.** Numa janela nova têm de ser definidas outra vez. Para as guardar para o seu utilizador do Windows (só valem em janelas abertas depois):
> `[Environment]::SetEnvironmentVariable("Jwt__Secret", "<segredo>", "User")`
> No cmd a sintaxe é `set "Jwt__Secret=<segredo>"`; no PowerShell, `set` não define variáveis de ambiente.

7. Criar o esquema e os dados iniciais

Há dois caminhos para o mesmo esquema (10 migrations, 17 tabelas, tipos enumerados, restrições e índices). O caminho recomendado é o 7.1 (migrations); o 7.3 dá o mesmo resultado só com SQL.

7.1. Aplicar as migrations (recomendado)

```
dotnet tool restore
dotnet ef database update --project src/Katch.Backend
```

O `dotnet tool restore` termina com `Restore was successful.` e só é preciso uma vez por cópia. O `dotnet ef database update` compila o projeto, mostra `Applying migration '...'` para cada uma das 10 migrations (de `20261015163645_InitialCreate` a `20261024180929_AddMessageAndOperationLog`) e termina com `Done.`

> **Mensagem esperada.** Na primeira vez aparece `Failed executing DbCommand ... FROM "__EFMigrationsHistory"`. Não é um erro: a tabela de histórico ainda não existia e é criada logo a seguir. Pode também aparecer várias vezes `Acquiring an exclusive lock for migration application`, que é informativo.

Exige `ConnectionStrings__Katch` e `Katch__AdminPassword`. Para confirmar:

```
psql -d katch -Atc "SELECT count(*) FROM information_schema.tables WHERE table_schema = 'public' AND table_type = 'BASE TABL
E' AND table_name <> '__EFMigrationsHistory';"
```

O resultado tem de ser `17`.

7.2. Carregar as listas pré-definidas e os dados de demonstração

Sem as listas pré-definidas não há localidades, e o registo de Candidatos e Empresas exige uma. Carregue-as sobre a base criada em 7.1:

```
psql -d katch -v ON_ERROR_STOP=1 -q -f database/02-seed-reference-data.sql
psql -d katch -v ON_ERROR_STOP=1 -q -f database/03-seed-demo-data.sql
psql -d katch -f database/04-verify.sql
```

Script	Conteúdo
`02-seed-reference-data.sql`	30 localidades com coordenadas, 30 competências e 12 benefícios (obrigatório para usar a API)
`03-seed-demo-data.sql`	Dados fictícios de demonstração (P18): 20 Candidatos ativos, 7 Recrutadores, 5 Empresas aprovadas e 2 pendentes, 50 vagas publicadas, respostas a vagas, 3 matches com conversa e o registo de autoria (opcional)
`04-verify.sql`	Só lê: conta as tabelas e os registos e confirma o P18

Resultado esperado do `04-verify.sql`: 17 tabelas do modelo, `existe_notification` igual a `f`, 20 Candidatos ativos, 5 Empresas aprovadas, 2 pendentes, 50 vagas publicadas, 1 Administrador e 0 resumos fora do formato. Os scripts 02 e 03 podem ser executados de novo sem erros nem linhas duplicadas.

Contas de demonstração. As contas do script 03 (`candidato01@demo.katch.pt` a `candidato20@demo.katch.pt` e `recrutador1@demo.katch.pt` a `recrutador7@demo.katch.pt`) partilham uma palavra-passe de demonstração que só existe como resumo nos scripts e **não é conhecida** (ver as notas de release, limitação L-09). Para as usar, defina uma palavra-passe sua com o procedimento da secção 7.5. Se não precisar delas, use o Administrador da secção 7.1 e crie contas novas pelos endpoints de registo (secção 9.2).

7.3. Alternativa: criar a base só com os scripts SQL

Numa base vazia, sem o `dotnet ef`:

```
psql -d katch -v ON_ERROR_STOP=1 -f database/01-create-schema.sql
psql -d katch -v ON_ERROR_STOP=1 -f database/02-seed-reference-data.sql
psql -d katch -v ON_ERROR_STOP=1 -f database/03-seed-demo-data.sql
psql -d katch -f database/04-verify.sql
```

O `01-create-schema.sql` foi gerado a partir das 10 migrations (`dotnet ef migrations script --idempotent`) e regista-as no `__EFMigrationsHistory`; por isso, um `dotnet ef database update` posterior não faz nada. A conta de Administrador criada por este script tem a palavra-passe de demonstração (resumo no script), que não é conhecida, e não a de `Katch__AdminPassword`: para iniciar sessão como Administrador, use o caminho 7.1 ou o procedimento da secção 7.5.

7.4. Repor a base

```
psql -d katch -v ON_ERROR_STOP=1 -f database/00-reset-schema.sql
```

Apaga todas as tabelas, tipos e dados (fica só o `__EFMigrationsHistory`, vazio). Depois volte a 7.1 ou 7.3. O mesmo efeito tem `dotnet ef database update 0 --project src/Katch.Backend`. Use só numa base local.

7.5. Definir uma palavra-passe para as contas de demonstração (opcional)

As palavras-passe só são guardadas como resumo (PBKDF2), por isso não se escrevem diretamente na base. O procedimento usa a própria API para gerar o resumo de uma palavra-passe à sua escolha e copia-o para as contas de demonstração. Com a API a correr (secção 8) e as listas pré-definidas carregadas:

1. No Swagger, em `POST /api/auth/candidates`, registe uma conta auxiliar com a palavra-passe pretendida (8 ou mais caracteres, com uma letra e um algarismo), por exemplo `definir.palavra@exemplo.pt`; `locationId` é um dos `id` de `GET /api/reference-lists/locations`. Resposta `201`.
2. Copie o resumo dessa conta para as contas de demonstração (e, se usou o caminho 7.3, para o Administrador). Guarde o texto seguinte num ficheiro `definir-palavra.sql` e execute-o:

   ```
   UPDATE app_user
   SET password_hash = (SELECT password_hash FROM app_user WHERE email = 'definir.palavra@exemplo.pt'),
       updated_at = now()
   WHERE email LIKE '%@demo.katch.pt' OR email = 'admin@katch.pt';
   ```

   ```
   psql -d katch -v ON_ERROR_STOP=1 -f definir-palavra.sql
   ```

   O `psql` responde `UPDATE 28` (20 Candidatos, 7 Recrutadores e o Administrador).
3. Inicie sessão com a palavra-passe escolhida: `candidato01@demo.katch.pt` com `X-Client-App: mobile`, `recrutador1@demo.katch.pt` (ou `admin@katch.pt`) com `X-Client-App: web`. Resposta `200`.

Se usou o caminho 7.1, retire `OR email = 'admin@katch.pt'` para manter a palavra-passe de `Katch__AdminPassword`. Só para uma base local de demonstração; o procedimento não faz parte do repositório.

8. Arrancar a API

```
dotnet run --project src/Katch.Backend --launch-profile http
```

Exige `ConnectionStrings__Katch` e `Jwt__Secret` na mesma janela. A API arrancou quando aparecer:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5180
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
```

A janela fica ocupada pela API; para os passos seguintes abra outra janela do PowerShell (sem precisar das variáveis) ou use o browser. Para parar a API, **Ctrl+C**.

- O perfil `http` define `ASPNETCORE_ENVIRONMENT=Development`, que é o ambiente em que o Swagger está disponível.
- Para HTTPS (`https://localhost:7180`), corra uma vez `dotnet dev-certs https --trust` e use `--launch-profile https`.
- As duas tarefas periódicas (reposição da quota de interesses e encerramento das vagas na data-limite) começam com a API e correm a cada 60 segundos.

9. Verificação

9.1. Documentação interativa (Swagger)

Abra `http://localhost:5180/swagger` no browser. Tem de aparecer a página *Katch.Backend* com os 13 controllers e os 72 endpoints da documentação da API v02 (secção 6), agrupados por `Auth`, `CandidateProfile`, `JobExploration`, `Company`, `Jobs`, `CandidateEvaluation`, `Matches`, `Conversations`, `Files`, `AdminAccounts`, `AdminCompanies`, `AdminIndicators` e `ReferenceLists`.

9.2. Experimentar a API

1. **Sem sessão:** em `GET /api/reference-lists/locations`, *Try it out* > *Execute*. Resposta `200` com as 30 localidades (se vier vazia, faltou o script 02).
2. **Início de sessão do Administrador:** em `POST /api/auth/login`, escolha `web` no cabeçalho `X-Client-App` e envie

   ```
   { "email": "admin@katch.pt", "password": "<valor de Katch__AdminPassword>" }
   ```

   Resposta `200` com `token`, `expiresAt` (8 horas depois), `userType` `ADMIN` e `userId`.
3. Clique em **Authorize** (cadeado, no topo), cole o `token` **sem** o prefixo `Bearer` e confirme.
4. `GET /api/admin/indicators`, com `from` e `to` (datas `yyyy-MM-dd`, obrigatórias, `to` não posterior a hoje; por exemplo `from=2026-10-01` e `to=2026-11-02`), devolve `200` com os sete indicadores (`activeCandidates`, `approvedCompanies`, ...). Sem as datas devolve `400` `ERR-10`; sem credencial, `401` `ERR-01`.
5. Para experimentar o fluxo de match com contas suas: registe um Candidato em `POST /api/auth/candidates` e um Recrutador em `POST /api/auth/recruiters` (o `locationId` é um dos `id` do passo 1); com a sessão do Recrutador (`X-Client-App: web`) registe a Empresa em `POST /api/recruiter/company`; com a sessão do Administrador aprove-a em `POST /api/admin/companies/{companyId}/approve`. O Candidato inicia sessão com `X-Client-App: mobile`. Respostas esperadas: `201` nos três registos; antes da aprovação, `GET /api/recruiter/jobs` devolve `403` `ERR-07` («Estado atual da Empresa: pendente»); a aprovação devolve `204`; depois dela, o mesmo pedido devolve `200`.

Os pedidos, as respostas e os códigos de erro de cada endpoint estão na documentação da API v02 (`m3-documentacao-api-v02.md`) e no README do repositório (secções 9 e 13).

9.3. Testes automáticos

Numa janela na raiz do repositório (com ou sem a API a correr):

```
dotnet test
```

Os testes de integração usam SQLite em memória: **não precisam do PostgreSQL nem de nenhuma variável de ambiente** (cada execução gera o seu segredo JWT). Na versão m3-v01 o resultado tem de ser:

```
Passed!  - Failed:     0, Passed:  1229, Skipped:     0, Total:  1229 - Katch.Backend.UnitTests.dll (net10.0)
Passed!  - Failed:     0, Passed:   771, Skipped:     0, Total:   771 - Katch.Backend.IntegrationTests.dll (net10.0)
```

Ou seja, **2000 testes** e 0 falhas. Os mesmos números constam da pipeline CI da main nesse commit (execução 41472193023 no GitHub Actions).

9.4. Verificações da pipeline (opcional)

Para repetir localmente o que a pipeline CI verifica em cada Pull Request:

```
dotnet build --configuration Release -warnaserror
dotnet format --verify-no-changes
dotnet test --configuration Release
```

O primeiro falha com qualquer aviso dos Roslyn Analyzers ou do StyleCop; o segundo falha se algum ficheiro não respeitar o `.editorconfig`.

10. Configuração opcional

O quê	Como
Intervalo das tarefas periódicas	`$env:BackgroundJobs__IntervalSeconds = "10"` antes do `dotnet run`. Fora de 1 a 86400, a API não arranca e diz porquê.
Pasta dos ficheiros enviados	`$env:FileStorage__RootPath = "C:\katch\files"`. A pasta é criada quando é preciso. Os ficheiros só são servidos pela API (`GET /api/files/...`), com autorização.
Limites dos ficheiros	Fixos no código, conforme a API v02 (secção 2.6): fotografia de perfil JPEG/PNG até 5 MB; CV PDF até 5 MB; logótipo JPEG/PNG até 2 MB; galeria da Empresa até 6 fotografias e vaga até 5, JPEG/PNG até 5 MB cada.
Credencial JWT	Só o segredo é configurável. Emissor `katch-backend`, destinatário `katch-clients` e validade de 8 horas são fixos.
Fora de `Development`	Com `ASPNETCORE_ENVIRONMENT` diferente de `Development`, a API funciona, mas o Swagger não é publicado.

11. Resolução de problemas

Na coluna «Ensaio», «Sim» indica que a mensagem foi reproduzida e copiada do ensaio do anexo A; «—» indica um caso descrito a partir do README do repositório ou do código, não reproduzido.

Erro ou sintoma	Causa	Solução	Ensaio
`No .NET SDKs were found`, ou `dotnet` não é reconhecido	Só o runtime está instalado, ou a janela é anterior à instalação	Instalar o SDK (3.1) e abrir uma janela nova	—
`dotnet` na pasta do repositório responde `The command could not be loaded, possibly because: ...`	SDK mais antigo do que o exigido pelo `global.json` (10.0.401), por exemplo 10.0.1xx ou 9.x	Instalar o SDK 10.0.401 ou superior (`dotnet --list-sdks` mostra os instalados)	Sim (simulado)
`git clone`: `Filename too long` e `unable to checkout working tree`	Caminho da pasta longo demais (limite de 260 caracteres do Windows)	Usar uma pasta curta, como `C:\katch`; em alternativa, `git config --global core.longpaths true` antes do clone	Sim
`Unhandled exception. System.InvalidOperationException: Falta a cadeia de ligação ao PostgreSQL. Defina a variável de ambiente ConnectionStrings__Katch.`	`ConnectionStrings__Katch` não definida nesta janela	Definir a variável (secção 6) e repetir	Sim
`Unhandled exception. Microsoft.Extensions.Options.OptionsValidationException: Falta o segredo JWT ou tem menos de 32 bytes. ...`	`Jwt__Secret` em falta ou curto	Definir um segredo com pelo menos 32 bytes (secção 6)	Sim
`dotnet ef database update`: `Falta a palavra-passe da conta de Administrador. Defina a variável de ambiente Katch__AdminPassword.`	`Katch__AdminPassword` não definida	Definir a variável e repetir	Sim
`A palavra-passe do Administrador deve ter pelo menos 8 caracteres ...`	Valor de `Katch__AdminPassword` fora do P05	Pelo menos 8 caracteres, com uma letra e um algarismo	—
`dotnet ef`: `Failed to connect to 127.0.0.1:6000`	`ConnectionStrings__Katch` não definida (o `dotnet ef` usa uma cadeia por omissão, sem credenciais)	Definir a variável e repetir	Sim
`Failed to connect to 127.0.0.1:<porto>`	Porto errado ou serviço do PostgreSQL parado	Confirmar o porto (pgAdmin > servidor > *Properties* > *Connection*) e o serviço `postgresql-x64-18` em *Serviços*	Sim
`28P01: password authentication failed for user "postgres"`	Palavra-passe ou utilizador errados na cadeia de ligação	Corrigir `ConnectionStrings__Katch`; é a palavra-passe definida na instalação	Sim

Pedidos à API devolvem `500` e o registo da API mostra `3D000: database "katch" does not exist`	A base não foi criada, ou o nome em `Database=` está errado	Criar a base (secção 5) ou aplicar as migrations (7.1): o `dotnet ef database update` cria a base se ela não existir	Sim
A API arranca, mas os pedidos devolvem `500` com `42P01: relation ... does not exist` no registo da API	A base existe, mas faltou criar o esquema	Secção 7.1 ou 7.3	—
`psql` ou `createdb` não é reconhecido	A pasta `bin` do PostgreSQL não está no PATH	`$env:Path = "C:\Program Files\PostgreSQL\18\bin;" + $env:Path`	—
`GET /api/reference-lists/locations` devolve `[]`; o registo recusa a localidade	Faltou o script 02	Secção 7.2	—
Competência «Outro» com acento recusada pelo CHECK `ck_candidate_custom_skill_label`	Base criada com a locale `C`	Criar a base com a locale predefinida (secção 5)	—
Início de sessão: `400` `ERR-10` com o campo `X-Client-App`	Cabeçalho `X-Client-App` em falta ou com outro valor	`web` para Administrador e Recrutador; `mobile` para Candidato	Sim
Início de sessão: `403` `ERR-05` («Esta conta não tem acesso por este ponto de acesso.»)	Ponto de acesso errado para o tipo de conta	Trocar o valor de `X-Client-App`	Sim
Pedidos autenticados devolvem `401` `ERR-01`	Credencial em falta, colada com o prefixo `Bearer` no *Authorize*, expirada (8 h) ou emitida antes de mudar o `Jwt__Secret`	Repetir o início de sessão e colar só o `token`	Sim
Recrutador recebe `403` `ERR-07` («A operação exige a Empresa aprovada. Estado atual da Empresa: …»)	A Empresa ainda não está aprovada	Aprovar a Empresa com o Administrador (9.2, passo 5)	Sim
`GET /api/admin/indicators`: `400` `ERR-10` nos campos `from` e `to`	Faltam as datas do período	Indicar `from` e `to` (9.2, passo 4)	Sim
`System.IO.IOException: Failed to bind to address http://127.0.0.1:5180: address already in use.`	Outra cópia da API já está a correr	Fechar a outra janela (Ctrl+C) ou terminar o processo `Katch.Backend`	Sim
`/swagger` devolve `404`	A API não está no ambiente `Development`	Arrancar com `--launch-profile http` (secção 8)	—
`A newer version of tool 'dotnet-ef' is available`	Aviso informativo	Ignorar: o projeto usa a versão fixada (10.0.13)	Sim

12. Resumo dos comandos

Objetivo	Comando (na raiz do repositório)
Instalar o `dotnet-ef` (uma vez)	`dotnet tool restore`
Compilar	`dotnet build`
Criar o esquema (migrations)	`dotnet ef database update --project src/Katch.Backend`
Listas pré-definidas	`psql -d katch -v ON_ERROR_STOP=1 -f database/02-seed-reference-data.sql`
Dados de demonstração	`psql -d katch -v ON_ERROR_STOP=1 -f database/03-seed-demo-data.sql`
Verificar a base	`psql -d katch -f database/04-verify.sql`
Repor a base	`psql -d katch -v ON_ERROR_STOP=1 -f database/00-reset-schema.sql`
Arrancar a API	`dotnet run --project src/Katch.Backend --launch-profile http`
Swagger	`http://localhost:5180/swagger`
Testes	`dotnet test` (1229 + 771 = 2000)

Anexo A — Ensaio num ambiente limpo

Os passos deste manual foram executados de raiz em 2026-11-02, sobre a tag `v0.3.0`. Foram feitos dois ensaios seguidos sobre o mesmo clone: o principal (passos 1 a 16) e um complementar (passos A a K), cada um com um servidor PostgreSQL novo. Um terceiro ensaio, sobre uma base criada pelo caminho 7.3, confirmou o procedimento da secção 7.5: antes do `UPDATE`, `401` `ERR-02`; depois, `200` para `candidato01`, `candidato20`, `recrutador1`, `recrutador7` e `admin@katch.pt`, e `401` com uma palavra-passe errada. Os registos completos, com os segredos e as credenciais mascarados, estão anexados à Issue I106 como evidência, com o registo da compilação a partir da cópia arquivada em `04.12-backend/m3-v01` (0 avisos, 0 erros, 1229 + 771 testes a passar).

Elemento	Valor
Máquina	Windows 11 Home 10.0.26200, Windows PowerShell 5.1
Isolamento	Clone novo do GitHub; caches NuGet (`NUGET_PACKAGES`, `NUGET_HTTP_CACHE_PATH`) e pasta da CLI (`DOTNET_CLI_HOME`) novas e vazias; servidor PostgreSQL 18 novo, criado com `initdb` (superutilizador `postgres` com palavra-passe, UTF8, locale predefinida); variáveis do projeto removidas antes de começar; segredos gerados aleatoriamente
Ferramentas	SDK .NET 10.0.402; PostgreSQL 18.3; Git 2.55; `dotnet-ef` 10.0.13 (restaurado pelo manifesto)
Código	`3f4a070946035440e8ad9c746b83a7b904478881` (`git describe`: `v0.3.0`), 0 ficheiros em falta depois do checkout
Resultados	**Todos os passos concluídos.** Compilação sem avisos nem erros (cerca de 15 s depois do restore NuGet); 10 migrations aplicadas, 17 tabelas, `has-pending-model-changes`: «No changes have been made to the model since the last migration»; scripts 02, 03 e 04 sobre a base das migrations com os valores esperados (secção 7.2); API em `http://localhost:5180` em cerca de 3 s, ambiente `Development`; Swagger `200`, 62 caminhos, 72 operações, 13 grupos; 30 localidades sem sessão; início de sessão do Administrador `200`; indicadores `200`; registo de Candidato e Recrutador e registo e aprovação da Empresa como em 9.2; `dotnet test`: 1229 + 771 testes, 0 falhas; caminho só com SQL (7.3) com os mesmos valores, segunda execução sem erros nem duplicados, e reposição (7.4) a deixar 0 tabelas; `dotnet build -c Release -warnaserror` e `dotnet format --verify-no-changes` sem erros; mensagens da secção 11 marcadas com «Sim».
Não exercitado	As ferramentas (SDK, PostgreSQL, Git) já estavam instaladas na máquina; a instalação pelo `winget` não foi repetida, mas os três pacotes foram confirmados com `winget show`. A base foi criada com `createdb` e não pelo pgAdmin.
Problema encontrado	Na primeira tentativa, numa pasta com mais de 120 caracteres, o checkout falhou com `Filename too long`. Repetido numa pasta curta, sem outras alterações. Daí o aviso da secção 4 e a linha da secção 11.